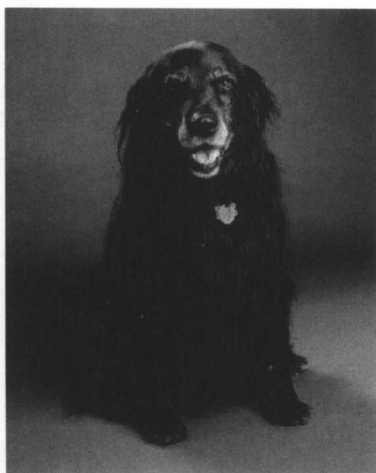


For Nancy,
without whom nothing
would be much worth doing

Wisdom and beauty form a very rare combination.

—— Petronius Arbiter
Satyricon, XCIV

And in memory of Persephone.
1995-2004



译序

按孙中山先生的说法，这个世界依聪明才智的先天高下得三种人：先知先觉得发明家，后知后觉得宣传家，不知不觉得实践家。三者之中发明家最少最稀珍，最具创造力。正是匠心独具的发明家创造了这个花花绿绿的计算机世界。

以文字、图书、授课形式来讲解、宣扬、引导技术的人，一般被视为宣传家而非发明家。然而，有一类最高等级的技术作家，不但能将精辟独到的见解诉诸文字，又能创造新的教学形式，引领风骚，对技术的影响和对产业的贡献不亚于技术或开发工具的创造者。这种人当之发明家亦无愧矣。

Scott Meyers 就是这一等级的技术作家！

自从 1991 年出版《Effective C++》之后，Meyers 声名大噪。1996 年的《More Effective C++》和 1997 年的《Effective C++》2/e 以及 2001 年的《Effective STL》让他更上高楼。Meyers 擅长探索编程语言的极限，穷尽其理，再以一支生花妙笔将复杂的探索过程和前因后果写成环环相扣故事性甚强的文字。他的幽默文风也让读者在高张力的技术学习过程中犹能享受“阅读的乐趣”——这是我对技术作家的最高礼赞。

以条款（items）传递专家经验，这种写作形式是否为 Meyers 首创我不确定，但的确是他造成了这种形式的计算机书籍写作风潮。影响所及，《Exceptional C++》、《More Exceptional C++》、《C++ Gotchas》、《C++ Coding Standards》、《Effective COM》、《Effective Java》、《Practical Java》纷纷在书名或形式上“向大师致敬”。

睽违 8 年之后《Effective C++》第三版面世了。我很开心继第二版再次受邀翻译。Meyers 在自序中对新版已有介绍，此处不待赘言。在此我适度修改第二版部分译序，援引于下，协助读者迅速认识本书定位。

C++ 是一个难学易用的语言！

C++ 的难学，不仅在其广博的语法，以及语法背后的语义，以及语义背后的深层思维，以及深层思维背后的对象模型；C++ 的难学还在于它提供了四种不同而又相辅相成的编程范型（programming paradigms）：procedural-based, object-based, object-oriented, generics。

世上没有白吃的午餐！又要有效率，又要弹性，又要前瞻望远，又要回溯相容，又要治大国，又要烹小鲜，学习起来当然就不可能太简单。在庞大复杂的机制下，万千使用者前仆后继的动力是：一旦学成，妙用无穷。

C++ 相关书籍车载斗量，如天上繁星，如过江之鲫。广博如四库全书者有之（*The C++ Programming Language*、*C++ Primer*、*Thinking in C++*），深奥如重山复水者有之（*The Annotated C++ Reference Manual*、*Inside the C++ Object Model*），细说历史者有之（*The Design and Evolution of C++*、*Ruminations on C++*），独沽一味者有之（*Polymorphism in C++*），独树一帜者有之（*Design Patterns*、*Large Scale C++ Software Design*、*C++ FAQs*），另辟蹊径者有之（*Generic Programming and the STL*），程序库大全有之（*The C++ Standard Library*），专家经验之累积亦有之（*Effective C++*、*More Effective C++*）。这其中“专家经验之累积”对已具 C++ 相当基础的程序员有着立竿见影的帮助，其特色是轻薄短小，高密度纳入作者浸淫 C++/OOP 多年的广泛经验。它们不但开展读者的视野，也为读者提供各种 C++/OOP 常见问题的解决模型。某些主题虽然在百科型 C++ 语言书中也可能提过，但此类书籍以深度探索的方式让我们了解问题背后的成因、最佳解法，以及其他可能的牵扯。这些都是经验的累积和心血的结晶，十分珍贵。

《*Effective C++*》就是这样一本轻薄短小高密度的“专家经验累积”。

本中译版与英文版页页对译，保留索引，偶尔加上小量译注；愿能提供您一个愉快的学习。千里之行始于足下，祝愿您从声名崇隆的本书展开一段新里程。同时，我也向您推荐本书之兄弟《*More Effective C++*》，那是 Meyers 的另一本同样盛名远播的书籍。

侯捷 2006/02/15 于台湾新竹

jjhou@jjhou.com

<http://www.jjhou.com>（繁体） <http://jjhou.csdn.net>（简体）

英中简繁术语对照

这里列出本书出现之编程术语的英中对照。本中文版在海峡两岸同步发行，因此我也列出本书简繁两版的术语对照，方便某些读者从中一窥两岸计算机用语。

表中带有 * 者表示本书对该词条大多直接采用英文术语。中英术语的选择系由以下众多考虑中取其平衡：

- 业界和学界习惯。即便是学生读者，终也要离开学校进入职场；熟悉业界和学界的习惯用语（许多为英文），避免二次转换，很有必要。
- 这是一本中文版，需顾及中文阅读的感觉和顺畅性。过多保留英文术语会造成版面的破碎与杂乱！然若适度保留英文术语，可避免某些望之不像术语的中文出现于字里行间造成阅读的困扰和停顿，有助于流畅的思考和留下深刻印象。
- 凡涉及 C++ 语言关键字之相关术语皆保留。例如 `class`, `struct`, `template`, `public`, `private`, `protected`, `static`, `inline`, `const`, `namespace` ……
- 以上术语可能衍生复合术语，例如与 `class` 相关的复合术语有 `base class`, `derived class`, `super class`, `subclass`, `class template` …… 此类复合术语如果不长，尽皆保留原文；若太长则视情况另作处理（也许中英并陈，也许赋予特殊字体）。
- 凡计算机科学所称之为数据结构名称，尽皆保留。例如 `stack`, `queue`, `tree`, `hashtable`, `map`, `set`, `deque`, `list`, `vector`, `array` …… 偶尔将 `array` 译为数组。
- 某些流通但不被我认为足够理想之中译词，保留原文不译。例如 `reference`。
- 某些英文术语被我刻意以特殊字体表现并保留，例如 *pass by reference*、*pass by value*、*copy* 构造函数、*assignment* 操作符、*placement new*。
- 少量术语为顾及词性平衡，时而采用中文（如指针、类型）时而采用英文（如 `pointer`、`type`）。
- 索引之于科技书籍非常重要。本书与英文版页页对译，因此原封不动保留所有英文索引。

过去以来我一直不甚满意 `object` 和 `type` 两个术语的中译词：“对象”和“类型”，认为它们缺乏术语突出性（前者正确性甚至有待商榷），却又频繁出现影响阅读，因此常在我的著作或译作中保留其英文词或偶尔采用繁体版术语：“物件”和“型别”。但现在我想，既然大家已经很习惯这两个中文术语，也许我只是杞人忧天。因此本书采用大陆读者普遍习惯的译法。不过我仍要提醒您，“`object`”在 `Object Oriented` 技术中的真正意义是“物体、物件”而非“对象、目标”。

以下带有 * 者表示本书对该词条采英文词，不译为中文

英文术语	简体版译词	繁体版译词
abstract	抽象的	抽象的
abstraction	抽象性、抽象件	抽象性、抽象件
access	访问	存取、取用
access level	访问级别	存取級別
access function	访问函数	存取函式
adapter	适配器	配接器
address	地址	地址
address-of operator	取地址操作符	取址運算子
aggregation	聚合	聚合
algorithm	算法	演算法
allocate	分配	配置
allocator	分配器	配置器
application	应用程序	應用程式
architecture	体系结构	體系結構
argument	实参	引數
*array	数组	陣列
arrow operator	箭头操作符	箭頭運算子
assembly language	汇编语言	組合語言
*assert(-ion)	断言	斷言
assign(-ment)	赋值	賦值
assignment operator	赋值操作符	賦值運算子
*base class	基类	基礎類別
*base type	基类型	基礎型別
binary search	二分查找	二分搜尋
*binary tree	二叉树	二元樹
binary operator	二元操作符	二元運算子
binding	绑定	綁定、繫結
*bit	位	位元
*bitwise	(以 bit 为单元逐一 ……)	
block	区块	區塊
boolean	布尔值	布林值
breakpoint	断点	中斷點
build	建置	建置
build-in	内置	內建
bus	总线	匯流排
*byte	字节	位元組
cache	高速缓存 (区)	快取 (區)
call	调用	呼叫
callback	回调	回呼
call operator	call 操作符	call 運算子

英文术语	简体版译词	繁体版译词
character	字符	字元
*child class	子类	子類別
*class	类	類別
*class template	类模板	類別模板
client	客户	客戶
code	代码	程式碼
compatible	兼容	相容
compile time	编译期	編譯期
compiler	编译器	編譯器
component	组件	組件
composition	复合	複合
concrete	具象的	具象的
concurrent	并发	並行
configuration	配置	組態
connection	连接	連接, 連線
constraint	约束 (条件)	約束 (條件)
construct	构件	構件
container	容器	容器
*const	(C++ 关键字, 代表 constant)	
constant	常量	常數
constructor	构造函数	建構式
*copy (动词)	拷贝	拷貝、複製
copy (名词)	复件、副本	複件、副本
create	创建	產生、建立、生成
custom	定制	訂制、自定
data	数据	資料
database	数据库	資料庫
data member	成员变量	成員變數
data structure	数据结构	資料結構
debug	调试	除錯
debugger	调试器	除錯器
declaration	声明式	宣告式
default	缺省	預設
definition	定义式	定義式
delegate	委托	委託
dereference	提领 (解参考)	提領
*derived class	派生类	衍生類別
design pattern	设计模式	設計範式
destroy	销毁	銷毀
destructor	析构函数	解構式
directive	指示符	指令
document	文档	文件
dynamic binding	动态绑定	動態綁定

英文术语	简体版译词	繁体版译词
entity	物体	物體
encapsulation	封装	封裝
*enum(-eration)	枚举	列舉
equality	相等	相等
equivalence	等价	等價
evaluate	核定、核算	核定、核算
exception	异常	異常
explicit	显式	顯式、明白的
expression	表达式	算式
file	文件	檔案
framework	框架	框架
full specialization	全特化	全特化
function	函数	函式
function object	函数对象	函式物件
*function template	函数模板	函式模板
generic	泛型、泛化、一般化	泛型、泛化、一般化
*getter (相对于 setter)	取值函数	取值函式
*global	全局的	全域的
*handle	句柄	識別號、權柄
*handler	处理函数	處理函式
*hash table	哈希表、散列表	雜湊表
header (file)	头文件	表頭檔
*heap	堆	堆積
hierarchy	继承体系 (层次结构)	繼承體系 (階層體系)
identifier	标识符	識別字、識別符號
implement(-ation)	实现	實作
implicit	隐喻的、暗自的、隐式	隱喻的、暗自的、隱式
information	信息	資訊
inheritance	继承	繼承
*inline	内联	行內
initialization list	初值列	初值列
initialize	初始化	初始化
instance	实体	實體
instantiate	具现化、实体化	具現化、實體化
interface	接口	介面
Internet	互联网	網際網路
interpreter	解释器	直譯器
invariants	恒常性	恒常性
invoke	调用	喚起
iterator	迭代器	迭代器
library	程序库	程式庫
linker	连接器	連結器
literal	字面常量	字面常數

英文术语	简体版译词	繁体版译词
* list	链表	串列
load	装载	載入
* local	局部的	區域的
lock	机锁	機鎖
loop	循环	迴圈
lvalue	左值	左值
macro	宏	巨集
member	成员	成員
member function	成员函数	成員函式
memory	内存	記憶體
memory leak	内存泄漏	記憶體洩漏
meta-	元-	超-
* meta-programming	元编程	超編程
modeling	塑模	模塑
module	模块	模組
modifier	修饰符	飾詞
multi-tasking	多任务	多工
* namespace	命名空间	命名空間
native	固有的	原生的
nested	嵌套	嵌套、巢狀
object	对象	物件
object based	基于对象的	植基於物件、以物件為基礎
object model	对象模型	物件模型
object oriented	面向对象	物件導向
operand	操作数	運算元
operating system	操作系统	作業系統
operator	操作符	運算子
overflow	溢出	上限溢位
overhead	额外开销	額外開銷
overload	重载	重載
override	覆写	覆寫
package	包	套件
parallel	并行	平行
parameter	参数、形参	參數
* parent class	父类	父類別
parse	解析	解析
partial specialization	偏特化	偏特化
* pass by reference	按址传递	傳址
* pass by value	按值传递	傳值
pattern	模式	範式
* placement delete	(某种特殊形式的 delete operator)	
* placement new	(某种特殊形式的 new operator)	
pointer	指针	指標